AI / ML PRACTICAL – COMPLETE PYTHON CODES

1. WEATHER FORECASTING EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
n = 100

df = pd.DataFrame({
    'Temperature': np.random.randint(15, 40, n),
    'Humidity': np.random.randint(30, 100, n),
    'WindSpeed': np.random.randint(0, 30, n),
    'Rainfall': np.random.randint(0, 50, n),
    'Pressure': np.random.randint(990, 1030, n)
})

print("Weather Dataset:")
print(df.head())
print("\nDataset Information:")
print(df.info())
print("\nStatistical Summary:")
print(df.describe())
print("\nMissing Values:")
print(df.isnull().sum())

df.hist(figsize=(10, 8))
plt.tight_layout()
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.title("Weather Correlation")
plt.show()

sns.pairplot(df)
plt.show()
```

2. PRODUCT SALES EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

df = pd.DataFrame({
    'Product': np.random.choice(
        ['Laptop', 'Mobile', 'Tablet', 'Headphone'], 100),
    'Quantity': np.random.randint(1, 20, 100),
    'Price': np.random.randint(500, 50000, 100)
})

df['Sales'] = df['Quantity'] * df['Price']

print(df.head())
print("\nStatistics:")
print(df.describe())
print("\nMissing Values:")
print(df.isnull().sum())

sns.barplot(data=df, x='Product', y='Sales')
plt.title("Product-wise Sales")
plt.xticks(rotation=30)
plt.show()

sns.histplot(df['Sales'], kde=True)
plt.title("Sales Distribution")
plt.show()

sns.heatmap(df[['Quantity', 'Price', 'Sales']].corr(),
            annot=True, cmap='coolwarm')
plt.show()
```

3. STUDENT MARKS EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

df = pd.DataFrame({
    'Maths': np.random.randint(30, 100, 50),
    'Science': np.random.randint(30, 100, 50),
    'English': np.random.randint(30, 100, 50),
    'Computer': np.random.randint(30, 100, 50)
})

print(df.head())
print("\nStatistics:")
print(df.describe())
print("\nMean Marks:")
print(df.mean())
print("\nMissing Values:")
print(df.isnull().sum())

df.mean().plot(kind='bar')
plt.title("Average Subject Marks")
plt.ylabel("Marks")
plt.show()

df.hist(figsize=(9, 7))
plt.tight_layout()
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

4. EMPLOYEE INCOME EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

df = pd.DataFrame({
    'Age': np.random.randint(22, 60, 100),
    'Experience': np.random.randint(1, 30, 100),
    'Income': np.random.randint(20000, 150000, 100)
})

print(df.head())
print(df.describe())
print(df.isnull().sum())

sns.histplot(df['Income'], kde=True)
plt.title("Employee Income Distribution")
plt.show()

sns.scatterplot(data=df, x='Experience', y='Income')
plt.title("Experience vs Income")
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

5. STOCK PRICE EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)
price = 100 + np.cumsum(np.random.randn(100))

df = pd.DataFrame({
    'Day': range(1, 101),
    'StockPrice': price,
    'Volume': np.random.randint(1000, 10000, 100)
})

print(df.head())
print(df.describe())
print(df.isnull().sum())

plt.plot(df['Day'], df['StockPrice'])
plt.title("Stock Price Trend")
plt.xlabel("Day")
plt.ylabel("Stock Price")
plt.show()

sns.histplot(df['StockPrice'], kde=True)
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

6. SENSOR DATASET EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

df = pd.DataFrame({
    'Temperature': np.random.uniform(20, 40, 100),
    'Humidity': np.random.uniform(30, 90, 100),
    'Pressure': np.random.uniform(990, 1030, 100),
    'GasLevel': np.random.uniform(10, 100, 100)
})

print(df.head())
print(df.describe())
print(df.isnull().sum())

plt.plot(df['Temperature'])
plt.title("Sensor Temperature Readings")
plt.show()

df.hist(figsize=(9, 7))
plt.tight_layout()
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

7. CUSTOMER INCOME EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

df = pd.DataFrame({
    'Age': np.random.randint(18, 65, 100),
    'Income': np.random.randint(15000, 150000, 100),
    'Spending': np.random.randint(1000, 50000, 100)
})

print(df.head())
print(df.describe())
print(df.isnull().sum())

sns.histplot(df['Income'], kde=True)
plt.title("Customer Income Distribution")
plt.show()

sns.scatterplot(data=df, x='Age', y='Income')
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

8. WEB TRAFFIC EDA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

np.random.seed(42)

df = pd.DataFrame({
    'Day': range(1, 101),
    'Visitors': np.random.randint(100, 2000, 100),
    'PageViews': np.random.randint(200, 5000, 100),
    'BounceRate': np.random.uniform(10, 90, 100),
    'SessionTime': np.random.uniform(1, 15, 100)
})

print(df.head())
print(df.describe())
print(df.isnull().sum())

plt.plot(df['Day'], df['Visitors'])
plt.title("Daily Website Visitors")
plt.xlabel("Day")
plt.ylabel("Visitors")
plt.show()

sns.histplot(df['PageViews'], kde=True)
plt.show()

sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```

9. MAP COLORING USING CSP

```
regions = ['A', 'B', 'C', 'D']
colors = ['Red', 'Green', 'Blue']

neighbors = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

assignment = {}

def is_valid(region, color):
    for neighbor in neighbors[region]:
        if neighbor in assignment:
            if assignment[neighbor] == color:
                return False
    return True

def solve(index):
    if index == len(regions):
        return True

    region = regions[index]

    for color in colors:
        if is_valid(region, color):
            assignment[region] = color
            if solve(index + 1):
                return True
            del assignment[region]

    return False

if solve(0):
    print("Map Coloring Solution:")
    for region, color in assignment.items():
        print(region, ":", color)
else:
    print("No solution found")
```

10. 8 QUEENS USING DFS

```
N = 8
board = [-1] * N

def is_safe(row, col):
    for i in range(row):
        if board[i] == col:
            return False
        if abs(board[i] - col) == abs(i - row):
            return False
    return True

def solve(row):
    if row == N:
        return True

    for col in range(N):
        if is_safe(row, col):
            board[row] = col
            if solve(row + 1):
                return True
            board[row] = -1
    return False

if solve(0):
    print("8 Queens Solution:")
    for i in range(N):
        for j in range(N):
            if board[i] == j:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No solution")
```

11. 8 QUEENS USING BFS

```
from collections import deque

N = 8

def is_safe(state, col):
    row = len(state)
    for r, c in enumerate(state):
        if c == col or abs(c - col) == abs(r - row):
            return False
    return True

def bfs():
    queue = deque([[]])
    while queue:
        state = queue.popleft()
        if len(state) == N:
            return state
        for col in range(N):
            if is_safe(state, col):
                queue.append(state + [col])
    return None

solution = bfs()
print("8 Queens BFS Solution:")

if solution:
    for row in range(N):
        for col in range(N):
            if solution[row] == col:
                print("Q", end=" ")
            else:
                print(".", end=" ")
        print()
else:
    print("No solution")
```

12. NAIVE BAYES CLASSIFIER

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X = [
    [30, 80], [25, 60], [35, 90],
    [20, 40], [28, 70], [32, 85],
    [22, 45], [33, 88], [24, 55],
    [36, 95], [21, 42], [29, 75]
]

# 1 = Play, 0 = Don't Play
y = [1, 1, 0, 0, 1, 0,
     0, 0, 1, 0, 0, 1]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Actual:", y_test)
print("Predicted:", predictions)
print("Accuracy:", accuracy_score(y_test, predictions))

new_data = [[28, 65]]
result = model.predict(new_data)

if result[0] == 1:
    print("Play")
else:
    print("Don't Play")
```

13. LINEAR AND LOGISTIC REGRESSION

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

# Linear Regression: Hours vs Marks
X = [[1], [2], [3], [4], [5],
     [6], [7], [8], [9], [10]]
y = [35, 40, 50, 55, 60, 65, 75, 80, 85, 95]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
prediction = linear_model.predict([[6]])

print("Linear Regression Prediction:")
print("Marks:", prediction[0])

# Logistic Regression: Pass or Fail
X2 = [[1], [2], [3], [4], [5],
      [6], [7], [8], [9], [10]]
y2 = [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]

X_train, X_test, y_train, y_test = train_test_split(
    X2, y2, test_size=0.2, random_state=42
)

log_model = LogisticRegression()
log_model.fit(X_train, y_train)
pred = log_model.predict(X_test)

print("\nLogistic Regression:")
print("Predictions:", pred)
print("Accuracy:", accuracy_score(y_test, pred))

result = log_model.predict([[7]])
print("Result:", "Pass" if result[0] == 1 else "Fail")
```

14. PRINCIPAL COMPONENT ANALYSIS (PCA)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

data = load_iris()
X = data.data
print("Original Dataset Shape:", X.shape)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
print("\nReduced Dataset:")
print(df.head())

print("\nExplained Variance:")
print(pca.explained_variance_ratio_)

plt.scatter(df['PC1'], df['PC2'])
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA Visualization")
plt.show()
```

15. ARTIFICIAL NEURAL NETWORK (ANN)

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

data = load_iris()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = MLPClassifier(
    hidden_layer_sizes=(10,),
    max_iter=1000,
    random_state=42
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)

print("Actual Values:")
print(y_test)
print("\nPredicted Values:")
print(predictions)

accuracy = accuracy_score(y_test, predictions)
print("\nAccuracy:", accuracy * 100, "%")

result = model.predict([[5.1, 3.5, 1.4, 0.2]])
print("\nPredicted Flower:", data.target_names[result[0]])
```

16. K-MEANS CLUSTERING

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

np.random.seed(42)

df = pd.DataFrame({
    'Income': np.random.randint(10000, 100000, 100),
    'Spending': np.random.randint(1000, 50000, 100)
})

print("Customer Dataset:")
print(df.head())

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

df['Cluster'] = kmeans.fit_predict(df)

print("\nCluster Results:")
print(df.head())

print("\nCluster Centers:")
print(kmeans.cluster_centers_)

plt.scatter(
    df['Income'],
    df['Spending'],
    c=df['Cluster']
)

plt.scatter(
    kmeans.cluster_centers_[:, 0],
    kmeans.cluster_centers_[:, 1],
    marker='X',
    s=200
)

plt.xlabel("Income")
plt.ylabel("Spending")
plt.title("Customer Segmentation using K-Means")
plt.show()
```